RankAlign Training Dynamics Analysis

When Does RankAlign / TC Improve? — **Per-Problem Fix**

Automated Analysis Report

June 21, 2026

Abstract

This report analyzes training dynamics for RankAlign models across four model/task combinations (gemma-2-9b-it and Qwen3.5-9B on membership and IFEval tasks). We characterize **(a)** when RankAlign improves correlation over the base model, **(b)** when the full method improves over basic RankAlign, and **(c)** when typicality correction (TC) specifically improves over non-TC training. We also report training diagnostics from WandB logs and additional statistics (concordance, score delta distributions).

This is the per-problem fix of the original report. The original computed each headline metric (gen ROC AUC, concordance, Spearman, Pearson) by pooling *all* candidates for a (model, task, setting, epoch) into one set and taking a single statistic, which conflates problems of differing difficulty and typicality baseline. Here every headline metric is computed **per problem, then averaged across problems, with a standard error** (SE = $\text{std}_{\text{ddof}=1}/\sqrt{n_{\text{problems}}}$). The problem unit is the IFEval *prompt* (79 train prompts, 40 candidates each) and the membership *category* (68 categories, 18–96 candidates each). Loss-based sections (WandB curves, loss-component breakdown, held-out validation loss) are unchanged — they were never pooled-ROC metrics.

1 Methodology: Per-Problem Aggregation

For each problem p (an IFEval prompt or a membership category) we take that problem's own candidate completions and compute the metric over them: gen ROC AUC ranks the problem's correct vs. incorrect candidates by the typicality-corrected generator score; concordance is the fraction of candidate pairs whose generator and validator score differences share sign; Spearman/Pearson correlate generator and validator scores across the problem's candidates. We then report the mean over problems with SE = $\text{std}_{\text{ddof}=1}/\sqrt{n_{\text{problems}}}$. This controls for the fact that different prompts/categories sit at different absolute score levels; pooling them yields a single global AUC that does not reflect within-problem ranking quality. The recomputation reuses the existing per-cell score files (which already carry the grouping column); no re-evaluation was needed. Pipeline: `analysis/scripts/build_report_fix_metrics.py` (reuses the file/setting matching from `analyze_all_combos.py`), reading `/datastor2/jdr/rankalign/outputs-trainset-dynamics/`

Effect of the fix. The membership numbers barely move (categories are already homogeneous), but the IFEval numbers rise substantially once prompts are no longer pooled (e.g. Gemma/IFEval base $0.670 \to 0.792$; Qwen/IFEval s4 $0.743 \to 0.858$): controlling for the per-prompt score baseline reveals stronger within-prompt ranking than the pooled AUC suggested. One ordering changes — on Gemma/IFEval, s3 and s4 are now statistically tied (Section 2).

Contents

2 Q1: When Does Each Component Help?

2.1 Main Comparison: Gen ROC AUC (TC, Self-Scoring)

Table 1 shows the generator ROC AUC (typicality-corrected) for the final model (epoch 2) evaluated on the training set with self-TC scoring, computed **per problem then averaged** (mean±SE over problems; columns: base, s1 SFT, s2 RA-basic, s3 RA-full, s4 TC-self). Bold = best per row.

Table 1: Generator ROC AUC (TC, self-scoring) at epoch 2, per-problem mean±SE. Bold = best per row. (Original pooled values: Gemma/Mem 0.851/0.941/0.948/0.976; Gemma/IFEval 0.670/0.586/0.482/0.753/0.801; Qwen/Mem 0.799/0.902/0.880/0.899/0.969; Qwen/IFEval 0.606/0.599/0.558/0.711/0.743.)

Model / Task	Base	s1	s2	s3	s4
Gemma / Membership	0.860 ±0.014	—	0.945 ±0.007	0.947 ±0.011	**0.974** ±0.009
Gemma / IFEval	0.792 ±0.018	0.646 ±0.025	0.491 ±0.022	**0.841** ±0.016	0.837 ±0.016
Qwen / Membership	0.806 ±0.016	0.908 ±0.011	0.880 ±0.011	0.896 ±0.008	**0.975** ±0.004
Qwen / IFEval	0.705 ±0.019	0.685 ±0.023	0.725 ±0.019	0.843 ±0.016	**0.858** ±0.015

2.2 Key Findings

1. **s4 (TC-self) is the best, or statistically tied for best, in every combination.** Under per-problem averaging it is strictly best in 3 of 4 cells; on Gemma/IFEval s3 (0.841 ±0.016) and s4 (0.837 ±0.016) are tied within one SE. (The original pooled numbers showed s4 strictly ahead everywhere; per problem the IFEval s3/s4 gap closes.)

2. **Basic RankAlign (s2) hurts on IFEval** — Gemma/IFEval drops from 0.792 (base) to 0.491 (s2, ≈chance). Without NLL constraints, generator scores explode.

3. **The full method (s3) consistently helps**, adding large gains over base on every combo (e.g. Gemma/IFEval 0.792 → 0.841, Qwen/IFEval 0.705 → 0.843).

4. **TC-self (s4) adds a further gain over s3 on 3 of 4 cells** (membership and Qwen/IFEval); on Gemma/IFEval s3 and s4 are tied — a consistent but, under per-problem averaging, no longer universal TC effect on gen ROC.

2.3 Visualizations

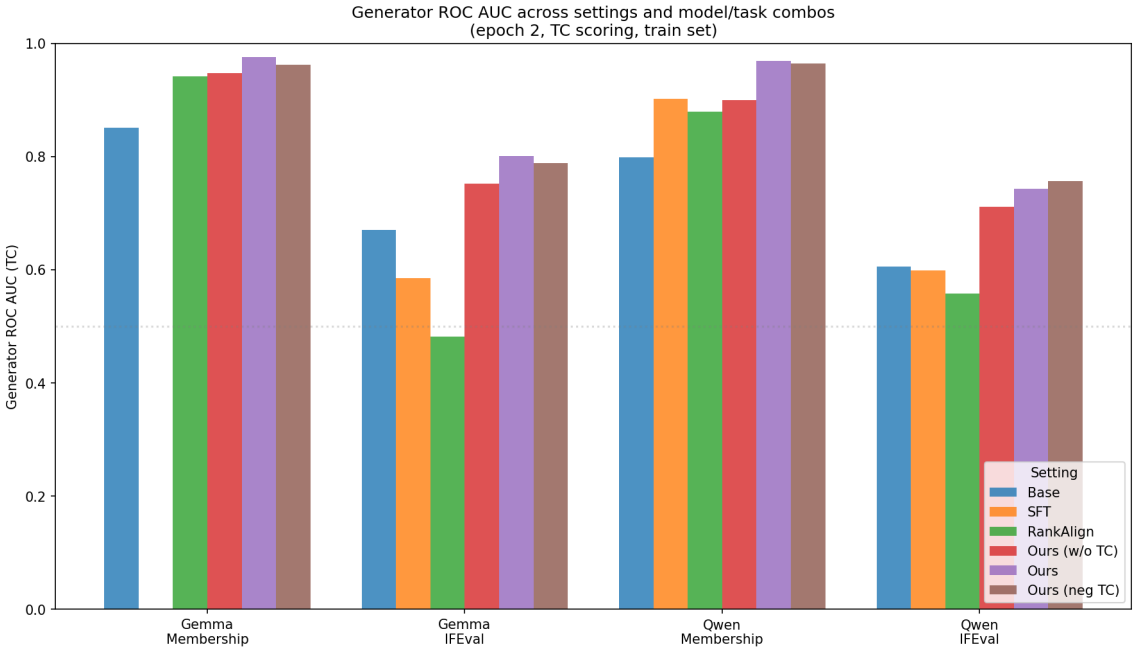

Figure 1: Generator ROC AUC across settings and model/task combinations (epoch 2, train set). Self-TC scoring is used for all settings except s7 (Ours, neg TC), which is scored with neg-TC at eval to match its training-time TC type.

3 Q2: Training Diagnostics

3.1 Loss Curves from WandB

Training was monitored via WandB for gemma-ifeval, qwen-membership, and qwen-ifeval. Gemma-membership was trained before WandB logging was added.

Smoothing. All loss curves in this section are smoothed using a running average with a window of 20 steps. To replicate in WandB: open a line plot panel's settings → Data tab → Smoothing Type: "Running Average" → Smoothing Parameter: 20. Raw (unsmoothed) curves are available directly on WandB.

3.1.1 IFEval

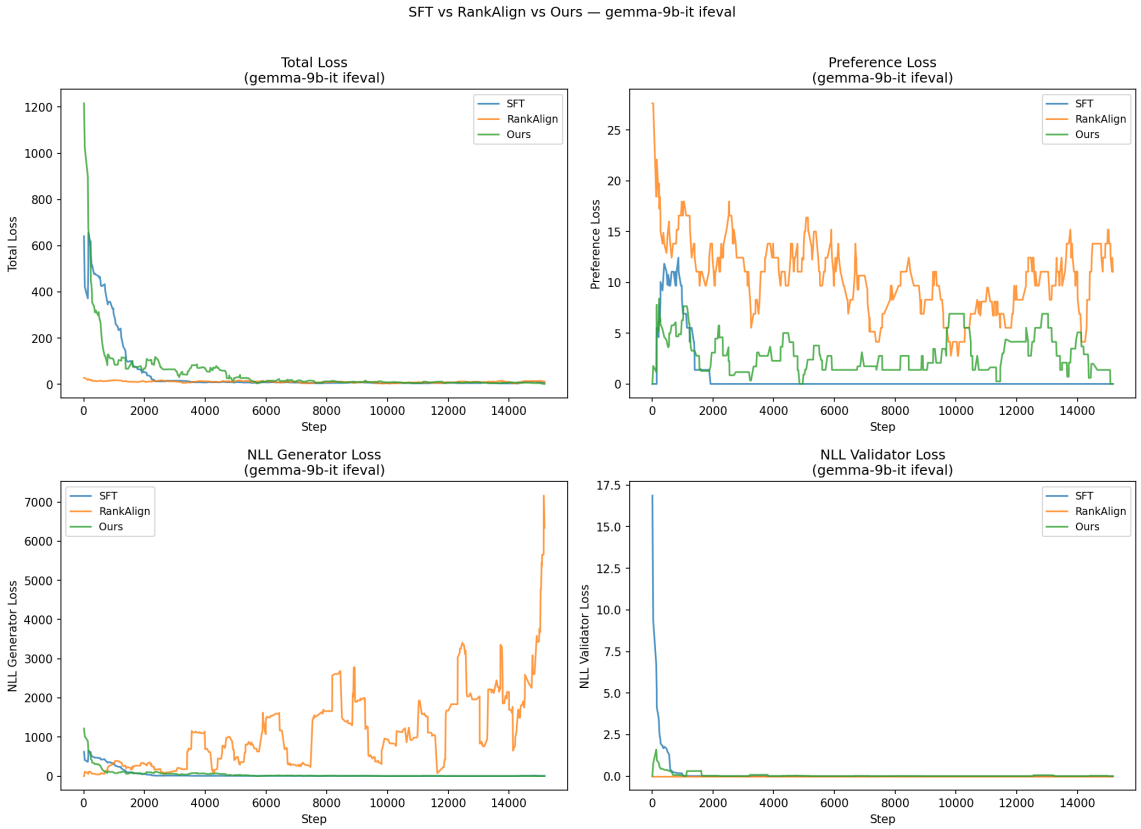

Figure 2: Loss curves: SFT vs RankAlign vs Ours (gemma-2-9b-it, IFEval).

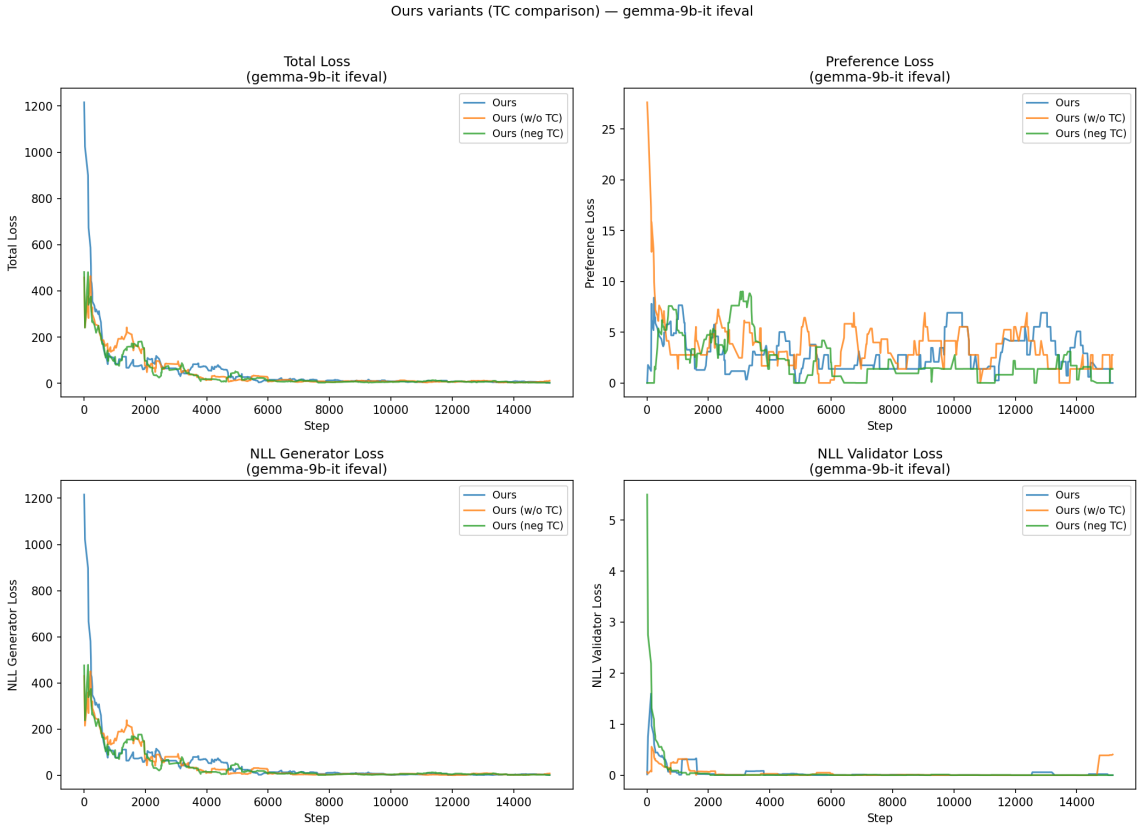

Figure 3: Loss curves: Ours TC comparison (gemma-2-9b-it, IFEval).

Figure 4: Loss curves: SFT vs RankAlign vs Ours (Qwen3.5-9B, IFEval).

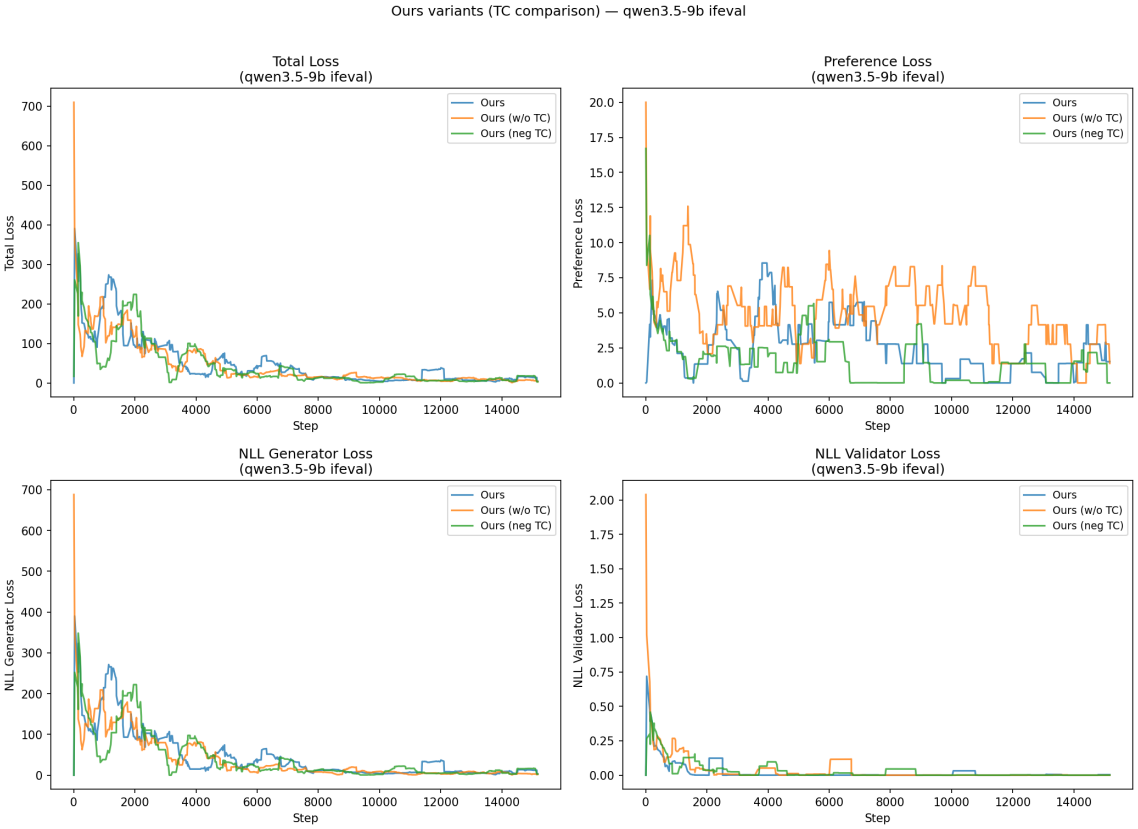

Figure 5: Loss curves: Ours TC comparison (Qwen3.5-9B, IFEval).

3.1.2 Membership

Note: gemma-2-9b-it membership was trained before WandB logging was added; no loss curves are available for that combination.

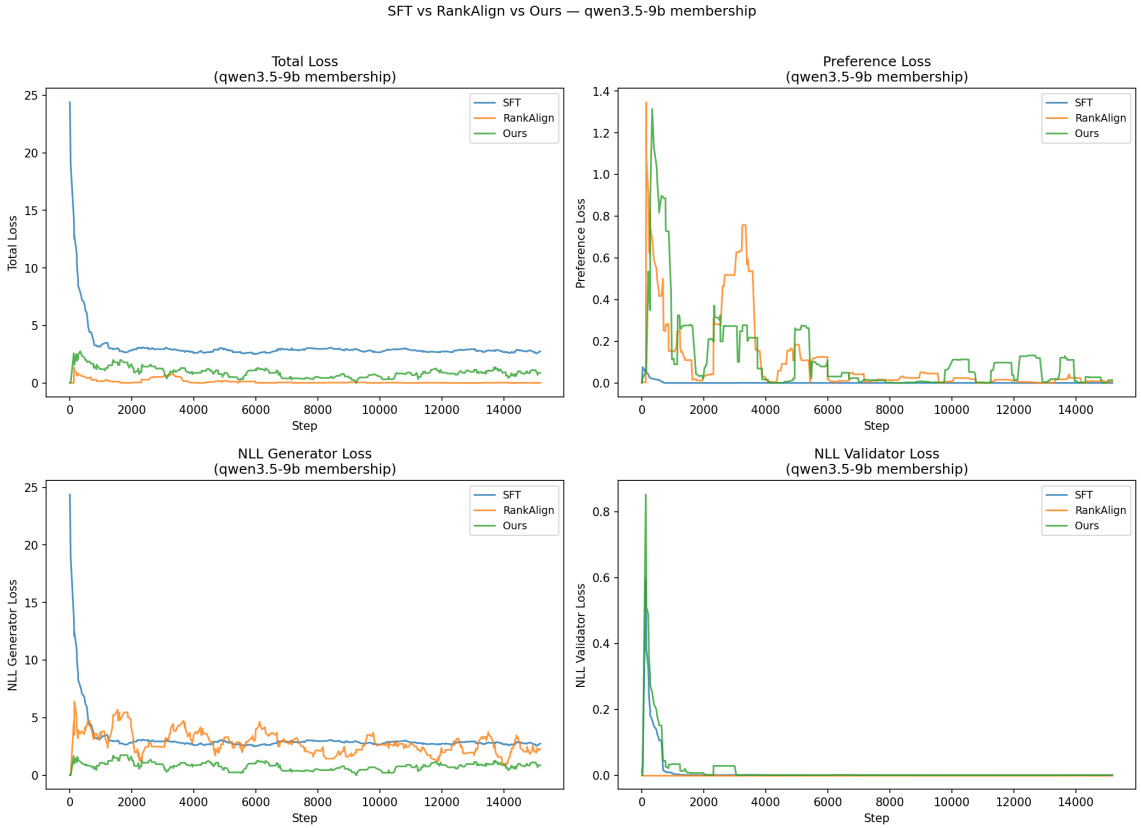

Figure 6: Loss curves: SFT vs RankAlign vs Ours (Qwen3.5-9B, membership).

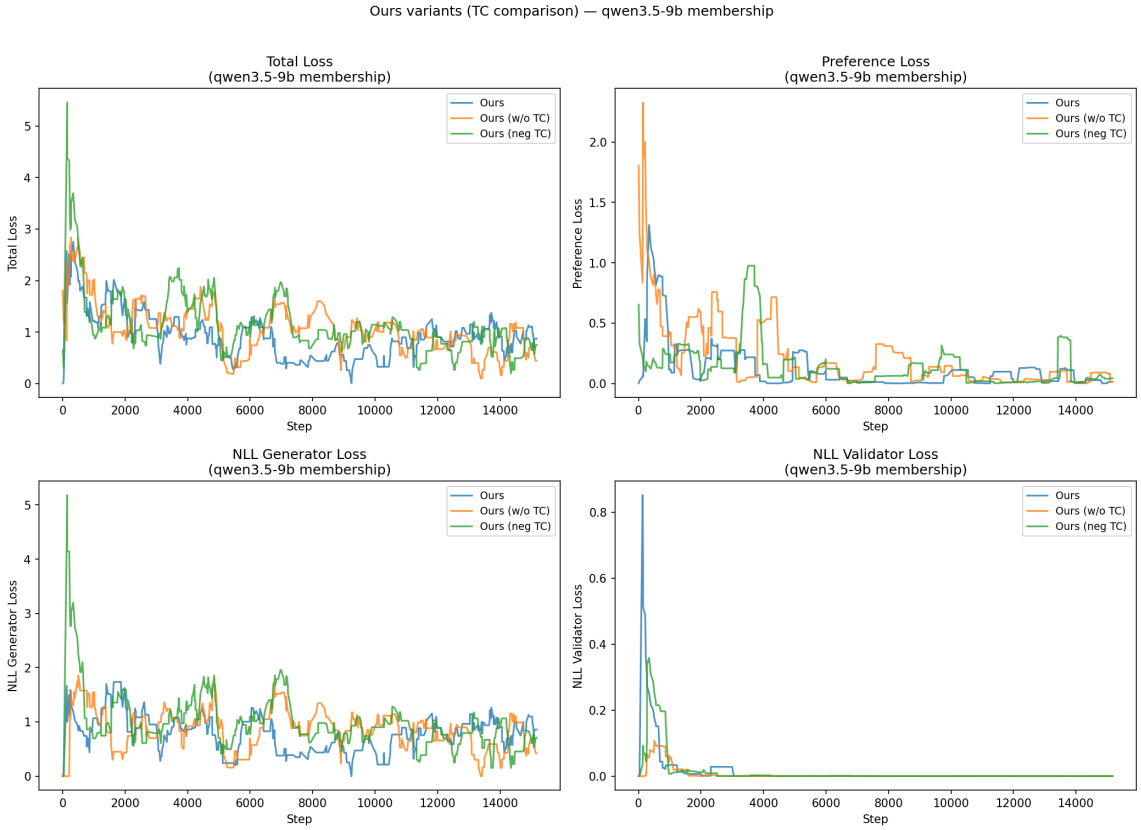

Figure 7: Loss curves: Ours TC comparison (Qwen3.5-9B, membership).

3.2 Loss Component Breakdown

Table 2 reveals the root cause of s2's failure on IFEval: without an NLL-G constraint, the generator's NLL explodes to **3120** (vs. 3.8–4.3 for constrained settings).

Table 2: Final-third loss components (Gemma IFEval). s2's NLL-G explosion explains its poor gen_roc.

Setting	**Total**	**Pref**	**NLL-G**	**NLL-V**
s1 (SFT)	5.99	0.00	5.99	0.000
s2 (basic)	**11.56**	**11.56**	**3119.8**	0.000
s4 (TC-self)	6.40	2.64	3.76	0.001
s3 (full)	6.15	1.81	4.33	0.009
s7 (TC-neg)	5.13	1.37	3.77	0.001

Key insight: On the simpler membership task (Qwen), s2 keeps NLL-G reasonable (2.42) without explicit constraint. The constraint is only critical for complex tasks where the optimization landscape allows score explosion.

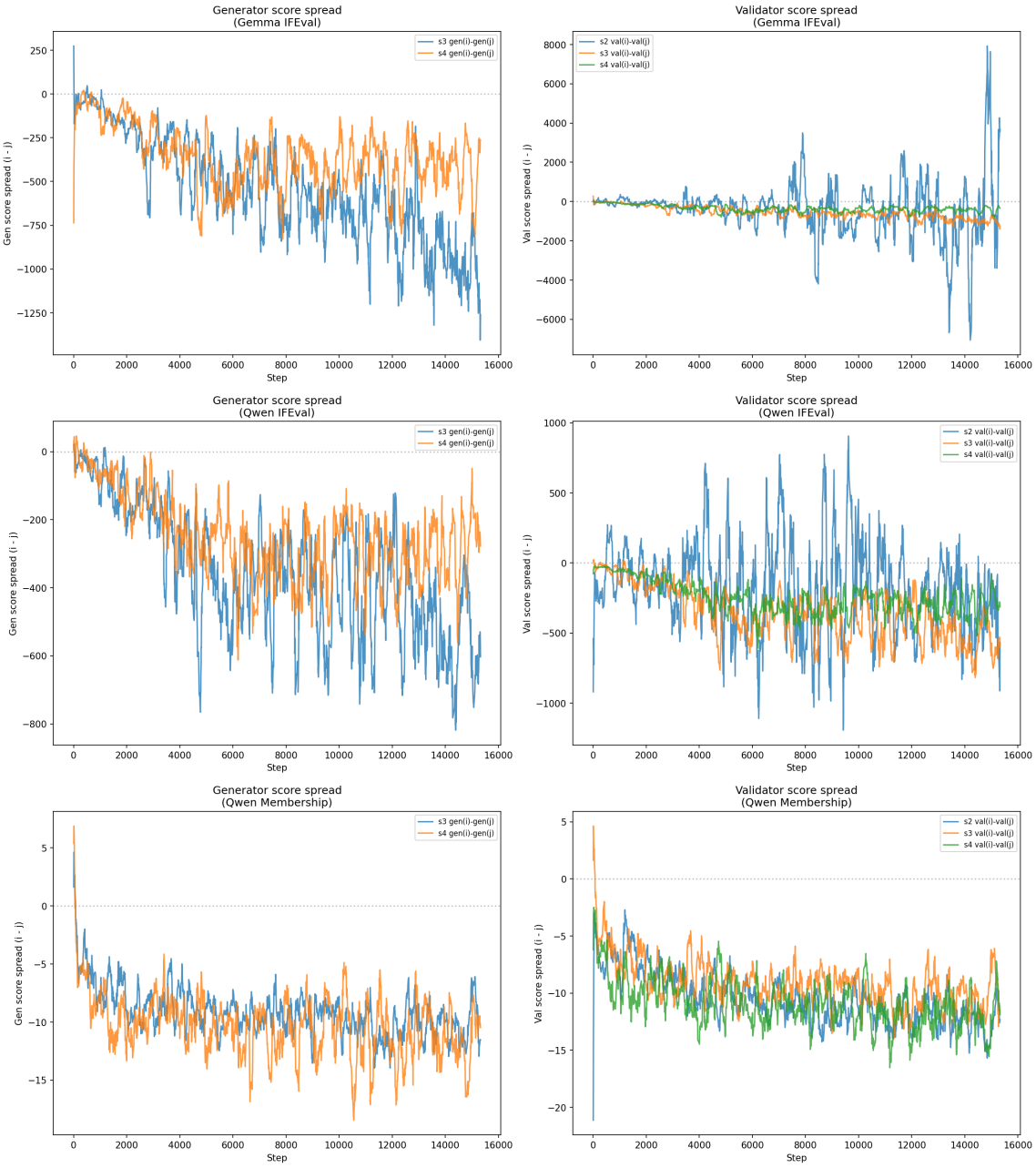

Figure 8: Score spread evolution during training. Each curve plots the difference between the two scores in a sampled training pair (item i minus item j) at each logged step, smoothed with a 20-step running average. Generator panels (left column) include only settings with $\mathrm{NLL}_G > 0$ (s2 with $\mathrm{NLL}_G = 0$ does not log per-item generator scores), so only s3 and s4 appear there; validator panels (right column) include s2, s3, s4. A more negative spread indicates a wider learned margin between the two paired items.

3.3 Held-out Validation Loss

The WandB curves above are training-set losses. To check whether each setting also reduces loss on *held-out* data, we recompute the same three loss components (preference, NLL-G, NLL-V) on a test set for each checkpoint (base, epoch 0, 1, 2). The base row is identical across settings (same untrained `gemma-2-9b-it`) and is shown as a dashed reference line in each panel.

Test sets.

- *Gemma / IFEval*: the held-out 50% of `ifeval-concat` (4,760 test items: 2,223 positive,

2,537 negative). 500 (positive, negative) pairs are sampled with a fixed seed.

- *Gemma / Membership*: `membership-sans-rosch-v0` is trained *without* rosch, so its held-out distribution is rosch itself. We aggregate `L_test` across all 11 registered `rosch-*` per-category tasks (1,042 items: 521 positive, 521 negative) and again sample 500 pairs.

The losses are computed with the same definitions used at training time: preference loss uses gen-score margins between the positive and negative item, NLL-G is the negative log-likelihood of the generator's gold completion, and NLL-V is the negative log-likelihood of the discriminator's correct yes/no token. Pairs and prompts are constructed by the same `make_prompt` the training script uses (membership models are evaluated under rosch's prompt template, which uses the same *"Complete the sentence: an example of . . . "* family as membership).

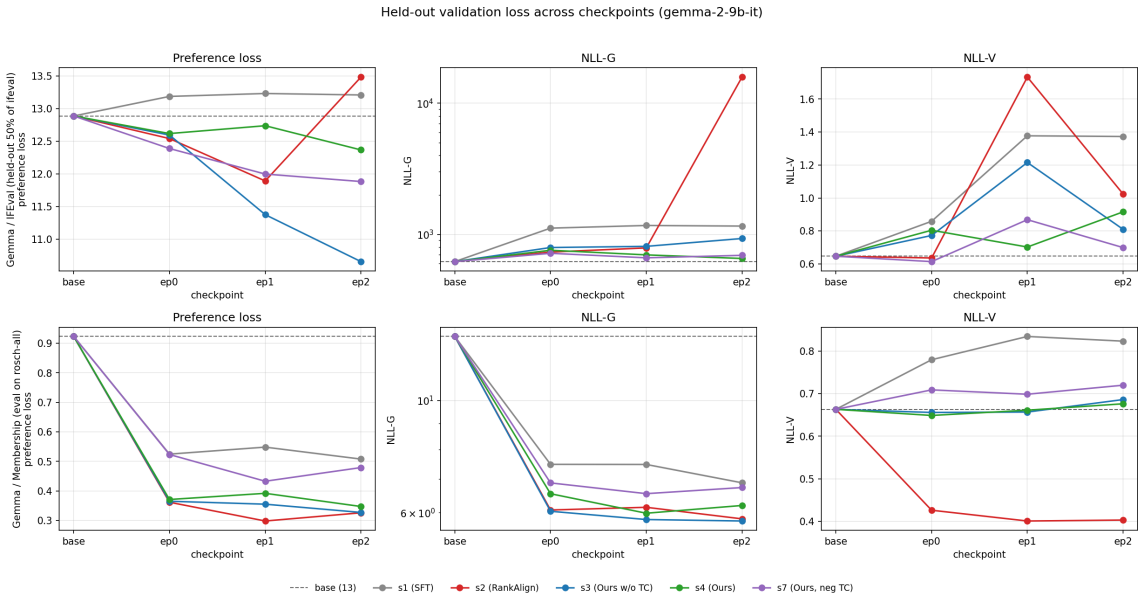

Figure 9: Held-out validation loss across checkpoints for gemma-2-9b-it. Top row: IFEval (held-out 50%). Bottom row: membership models evaluated on the aggregated rosch test set. The NLL-G column uses a log y-axis because s2 IFEval diverges. Dashed black line is the untrained base; markers at `ep0/ep1/ep2` show the three saved checkpoints from each setting.

Reading the figure.

- On membership/rosch (bottom row), every setting reduces all three losses substantially below base, and the orderings broadly match the train-set ranking (s3 and s2 lowest NLL-G; s4 and s3 lowest preference loss). This is the regime where RankAlign generalizes cleanly.

- On IFEval (top row), the picture is much harder. Preference loss only drops materially for s3 (12.89 $\rightarrow$ 10.66); s1 and s2 *rise* above base. The NLL-G panel is the headline: s2's generator NLL explodes to roughly 1.6×10^4 by epoch 2 on held-out IFEval, mirroring the train-side observation reported in Section 2.2. The constrained settings (s4, s3, s7) keep NLL-G in the same order of magnitude as base and are the only ones whose held-out preference loss does not regress.

- NLL-V tells a related story: s2 spikes the validator NLL on IFEval at epoch 1 before partially recovering, consistent with the score-explosion diagnostic — the validator becomes badly miscalibrated before it can re-stabilize. On membership, by contrast, s2 drives NLL-V down most aggressively (the validator becomes very confident there).

This held-out view confirms what the train-side curves and the Q1 ROC numbers already imply: *basic RankAlign without NLL constraints (s2) is unsafe on the harder IFEval task, even on data drawn from the same distribution* — it is not just an in-sample optimization artifact.

Reproducibility. The full pipeline lives at commit `3b71d4ac` on branch `longform`. Files involved:

- **Compute (Python)**: `scripts/compute_val_loss.py`. Loads the base model from `--model` plus the three merged epoch checkpoints found under `--models-dir` matching the `--setting` pattern; loads test data via `src/utils.py`'s `get_task` (with `import tasks` side-effect to register all task modules). For each checkpoint it computes the same three loss components used at training time and writes one row per (setting, checkpoint) to the CSV at `--output` (appending). Defaults: `--max-pairs 500`, `--seed 42`.
- **Special test-set logic for membership**: when `--test-task rosch-all` is passed, `compute_val_loss.py` aggregates `L_test` across every registered `rosch-*` task (1,042 items) and uses rosch's `make_prompt`/`get_label` for prompt construction. For ifeval, no `--test-task` is needed — the task itself returns a held-out split.
- **Launcher (Slurm)**: `scripts/run_val_loss.sh`. Reads env vars `MODEL`, `TASK`, `TEST_TASK`, `MODELS_DIR`, `SETTINGS`, `OUTPUT`, `HOURS`, exports `HF_HOME=/datastor2/jdr/.cache/huggingface` so all jobs reuse the existing `gemma-2-9b-it` snapshot (otherwise five concurrent jobs each redownload ∼36 GB and blow the home-filer quota), then submits one Slurm job per setting via `~/.local/bin/run 1 $HOURS --mem 64G ....`
- **Exact commands run** (from the repo root, `/datastor1/jdr/gv-gap/rankalign`, with `venv_lexcons` activated):

  ```
  # IFEval val-loss (5 jobs: 44826-44830, ~1h52m each on 1 GPU)
  TASK=ifeval-concat \
  OUTPUT=analysis/tables/val_loss_gemma_ifeval.csv \
  bash scripts/run_val_loss.sh

  # Membership val-loss, evaluated on rosch (5 jobs: 44831-44835, ~14m each)
  TASK=membership-sans-rosch-v0 \
  TEST_TASK=rosch-all \
  OUTPUT=analysis/tables/val_loss_gemma_mem.csv \
  bash scripts/run_val_loss.sh
  ```

  Both runs use the default `MODEL=google/gemma-2-9b-it`, `MODELS_DIR=/datastor2/jdr/rankalign/mod` and `SETTINGS="s1 s2 s3 s4 s7"`.
- **Output CSVs** (one row per setting/checkpoint, columns: `checkpoint, setting, model, task, n_pos, n_neg, n_pairs, preference_loss, nll_generator_loss, nll_validator_loss, total_loss, mean_gen_score_pos, mean_gen_score_neg, mean_val_score_pos, mean_val_score_neg`)
  - `analysis/tables/val_loss_gemma_ifeval.csv`
  - `analysis/tables/val_loss_gemma_mem.csv`
- **Plot (Python)**: `analysis/scripts/plot_val_loss.py`, run as

  ```
  python analysis/scripts/plot_val_loss.py
  ```

  This reads the two CSVs above (paths hard-coded relative to repo root) and writes `analysis/plots/val_loss_curves.png`, the figure shown above. NLL-G uses a log y-axis; preference loss and NLL-V use linear.

3.4 Generator-Validator Concordance

Concordance measures the fraction of item pairs where the generator and validator agree on the ordering direction.

Table 3: Concordance at epoch 2 (self-TC scoring), per-problem mean±SE. Chance = 0.5.

Model / Task	Base	s1	s2	s3	s4
Gemma / Membership	0.716 ±0.010	—	0.785 ±0.006	0.805 ±0.005	**0.826** ±0.006
Gemma / IFEval	0.654 ±0.010	0.598 ±0.013	0.460 ±0.013	0.748 ±0.009	**0.757** ±0.007
Qwen / Membership	0.680 ±0.009	0.740 ±0.007	0.753 ±0.006	0.768 ±0.005	**0.835** ±0.005
Qwen / IFEval	0.600 ±0.012	0.594 ±0.012	0.596 ±0.010	0.742 ±0.008	**0.750** ±0.008

Note: s2 on Gemma IFEval has per-problem concordance 0.460 (below the 0.5 chance line); the anti-correlation between generator and validator is clearer in the per-problem Spearman, where s2 on Gemma/IFEval is negative (-0.113 ± 0.037).

3.5 Spearman Correlation (Generator vs Validator)

Table 4: Spearman ρ between generator and validator scores (epoch 2, self-TC), per-problem mean±SE.

Model / Task	Base	s1	s2	s3	s4
Gemma / Membership	0.595 ±0.024	—	0.763 ±0.014	0.803 ±0.011	**0.839** ±0.008
Gemma / IFEval	0.439 ±0.026	0.282 ±0.036	−0.113 ±0.037	0.670 ±0.021	**0.697** ±0.015
Qwen / Membership	0.508 ±0.024	0.668 ±0.017	0.687 ±0.014	0.725 ±0.011	**0.855** ±0.007
Qwen / IFEval	0.290 ±0.033	0.273 ±0.035	0.272 ±0.029	0.661 ±0.020	**0.680** ±0.018

3.6 Pearson Correlation (Generator vs Validator)

Table 5: Pearson r between generator and validator scores (epoch 2, self-TC), per-problem mean±SE.

Model / Task	Base	s1	s2	s3	s4
Gemma / Membership	0.594 ±0.022	—	0.802 ±0.014	0.827 ±0.011	**0.897** ±0.006
Gemma / IFEval	0.453 ±0.030	0.271 ±0.041	−0.107 ±0.038	**0.762** ±0.022	0.736 ±0.016
Qwen / Membership	0.538 ±0.023	0.702 ±0.017	0.716 ±0.014	0.739 ±0.011	**0.876** ±0.004
Qwen / IFEval	0.297 ±0.034	0.291 ±0.037	0.280 ±0.032	**0.765** ±0.021	0.706 ±0.019

Pearson tracks Spearman on the membership tasks (s4 highest). On the IFEval tasks, s3 has higher Pearson than s4 even though s4 has higher Spearman — s4 preserves rank order while compressing absolute magnitudes via TC.

3.7 Score Delta Distributions

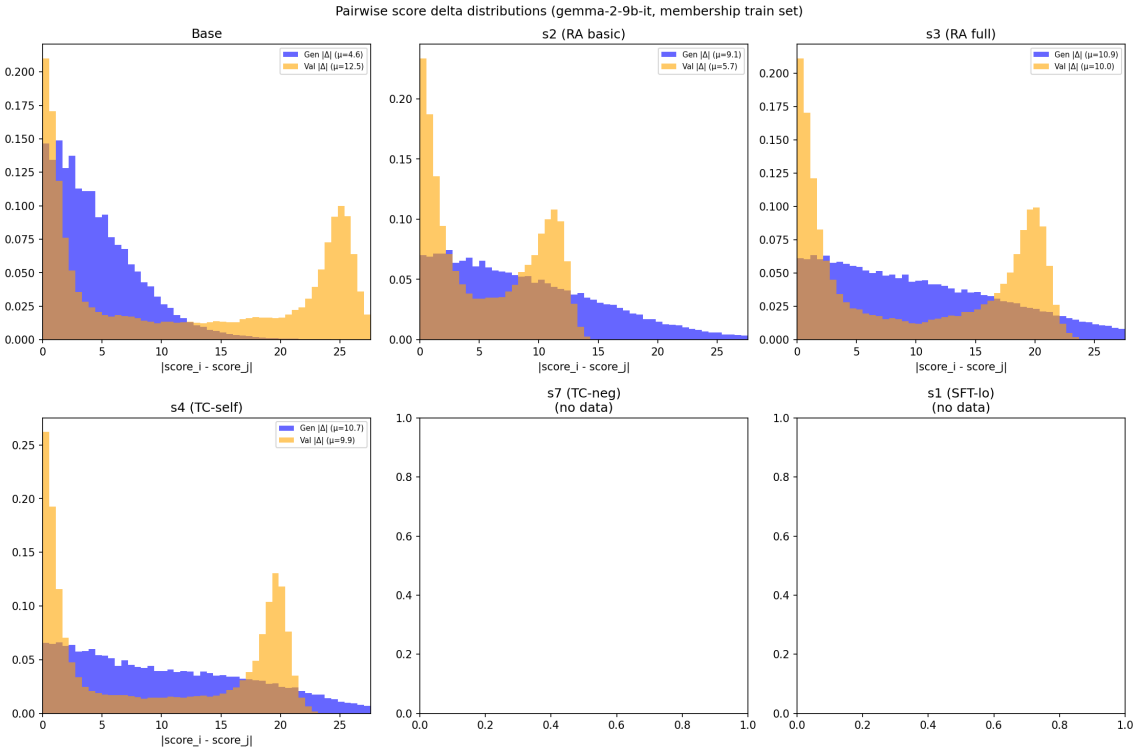

Figure 10: Pairwise score delta distributions for gemma membership. Generator (blue) and validator (orange) distributions. Tighter = better calibrated.

3.8 Training Dynamics

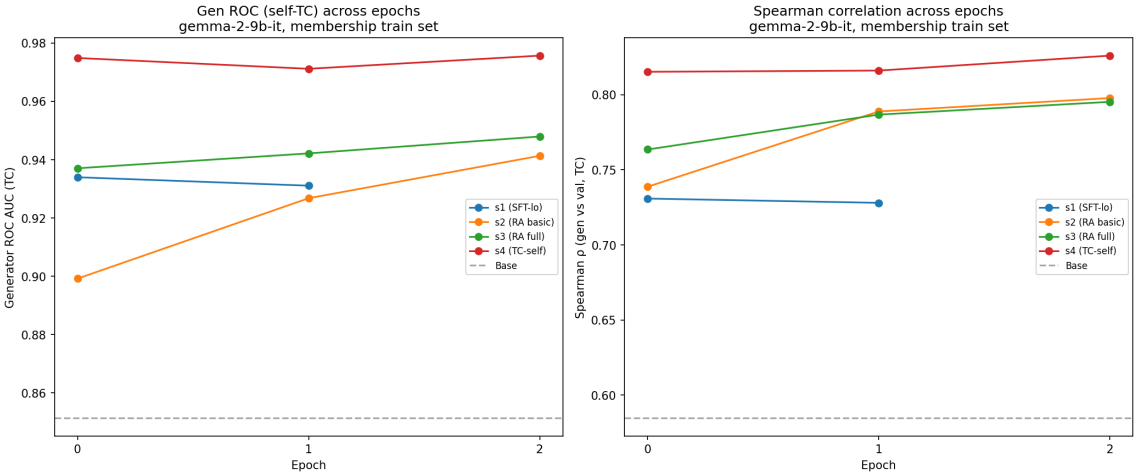

Figure 11: Training dynamics for gemma membership: Gen ROC and Spearman across epochs.

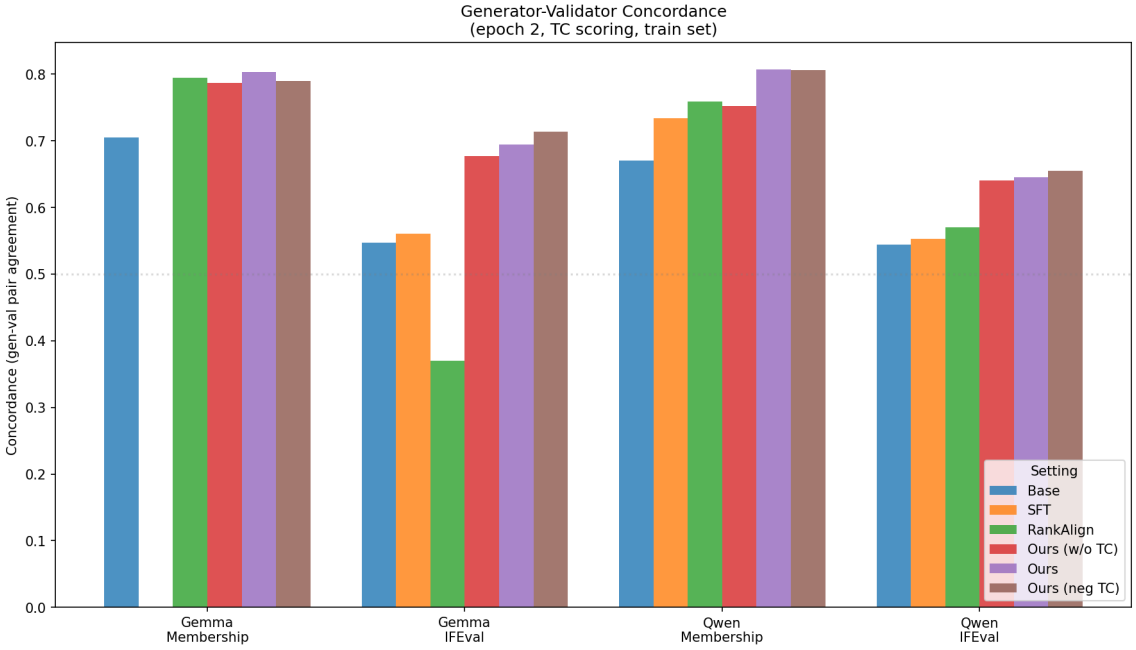

Figure 12: Concordance across all model/task combinations.

3.9 Generator-Validator Scatter Plots

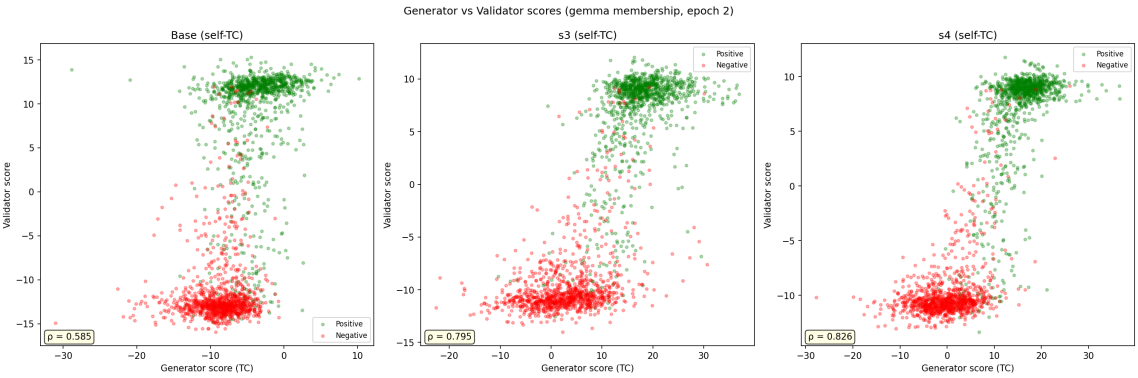

Figure 13: Generator vs validator scores for Base, s3, and s4 (self-TC scoring, gemma membership). Training improves the separation between positive (green) and negative (red) items in both generator and validator space.

3.10 Per-Category Analysis

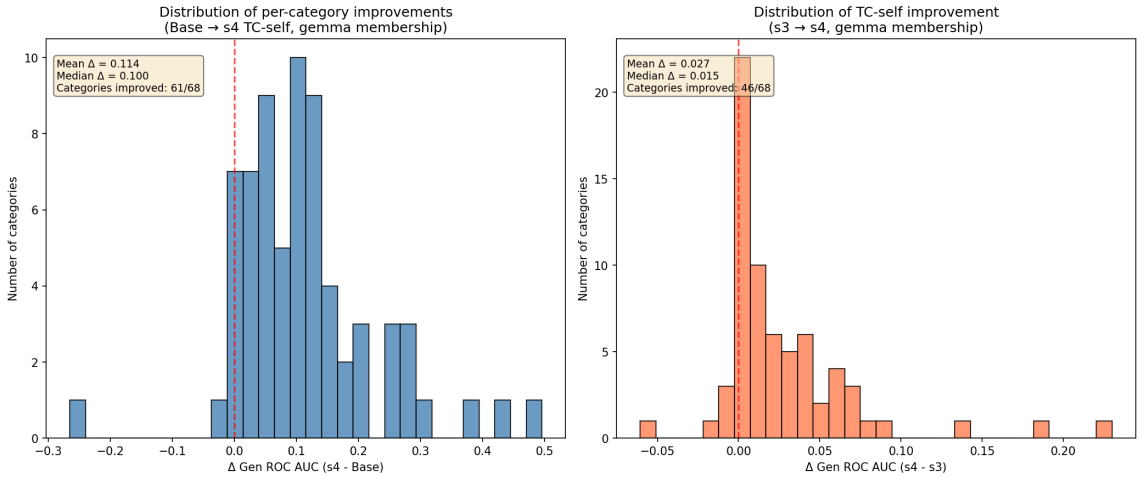

Figure 14: Left: distribution of per-category improvements (Base → s4). Right: improvements from TC-self specifically (s3 → s4). Both are consistently positive.

Categories where the base model was weakest (gen_roc ≈ 0.5–0.7) benefit most from training. For example, "medical specialty" improves from 0.504 to 1.000 (+0.496). Categories already near ceiling show minimal gains (expected).

4 Focused TC Comparison

4.1 Self-TC vs No-TC (s4 vs s3)

When evaluated with self-TC scoring, the typicality-corrected models consistently outperform the non-TC baseline (s3):

- s4 (TC-self, full): +0.028 over s3
- s6 (TC-self fsx, low δ): +0.023 over s3
- s11 (TC-self vlo, no fsx/ppd): +0.022 over s3
- s5 (TC-self basic): +0.010 over s3

4.2 Neg-TC vs No-TC (s7 vs s3)

When evaluated with neg-TC scoring, the improvement from TC-neg training is even larger:

- s12 (TC-neg vlo): +0.075 over s3
- s7 (TC-neg, full): +0.072 over s3

This suggests that TC-trained models learn better internal representations that are specifically captured by the matching TC scoring method.

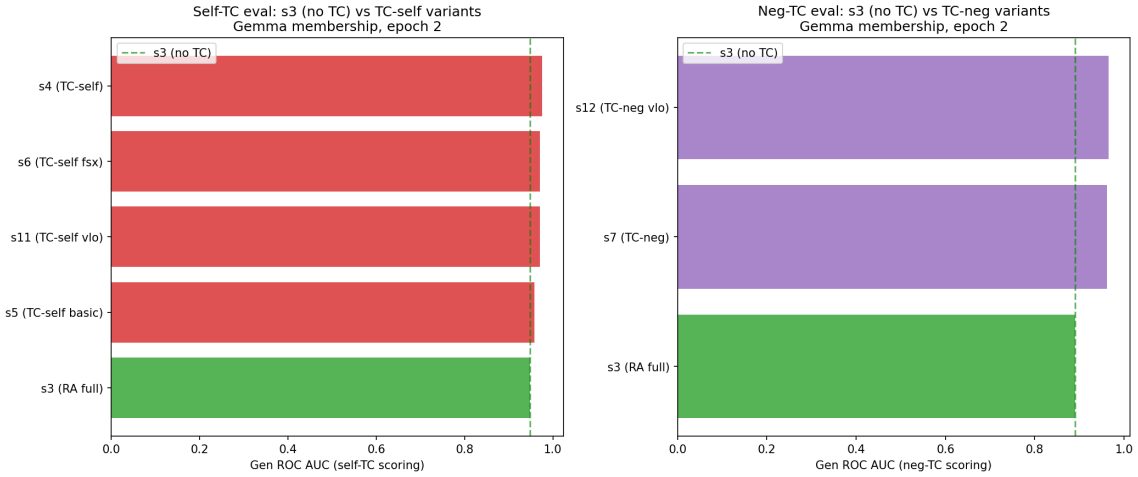

Figure 15: TC comparison: self-TC variants (left) and neg-TC variants (right) against s3 baseline.

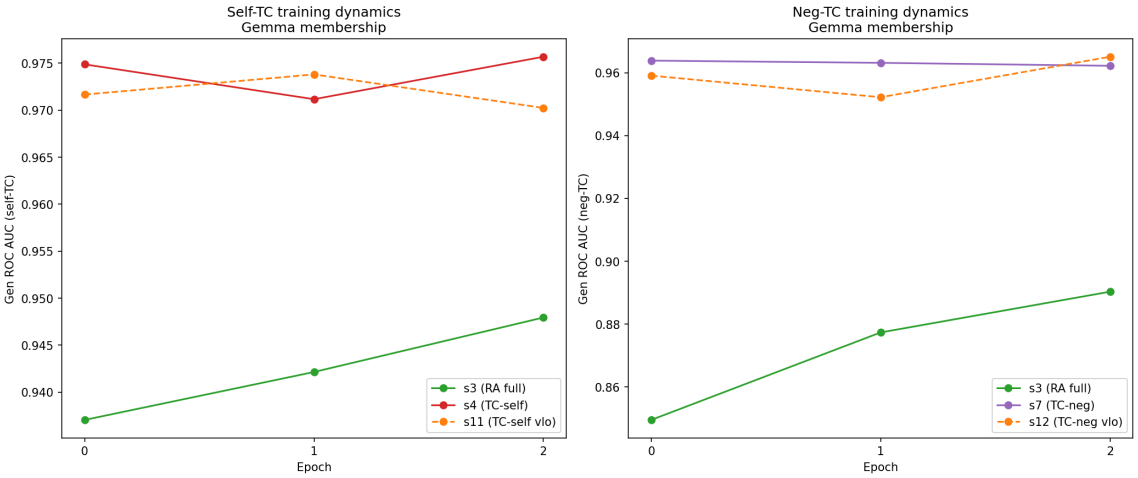

Figure 16: Epoch-by-epoch dynamics for TC comparison. TC-trained models converge quickly and maintain advantage throughout training.

5 Train vs Test Generalization

A critical finding emerges when comparing train-set dynamics with held-out test sets.

Table 6: Train vs test gen_roc: gemma-2-9b-it membership/rosch (basetyp scoring).

Setting	**Train (membership)**	**Test (rosch)**	**Δ**
s11 (TC-self vlo)	0.970	**0.924**	−0.046
s4 (TC-self full)	**0.976**	0.920	−0.056
s5 (TC-self basic)	0.958	0.913	−0.045
s2 (RA basic)	0.941	0.891	−0.050
s3 (RA full)	0.948	0.885	−0.063

Key finding: s3 (full method, no TC) outperforms s2 on training categories but *underperforms* on test categories (different Rosch categories). Meanwhile, all TC-trained models (s4,

s11, s5) maintain their advantage on both splits. This suggests **TC promotes generalization** — it prevents overfitting to training-specific patterns.

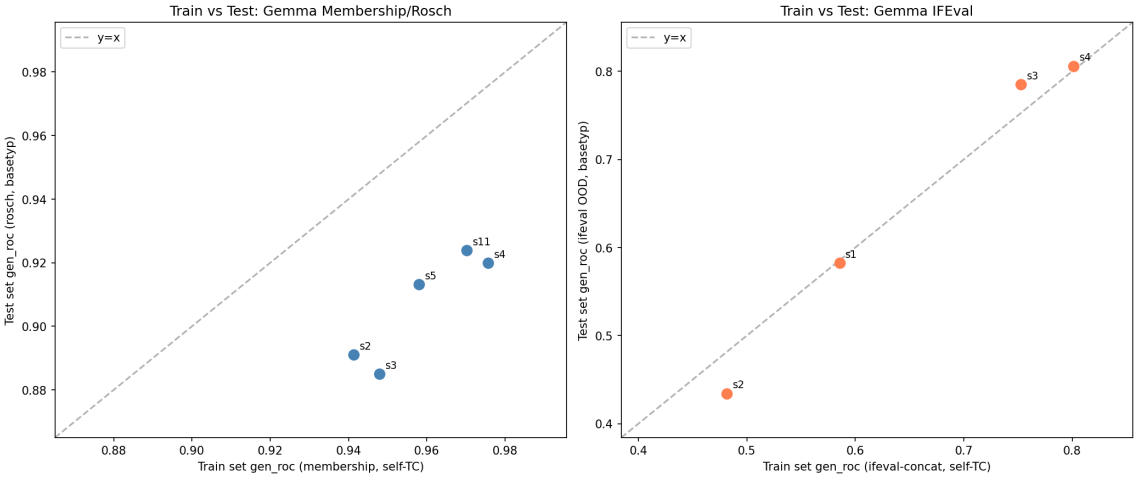

Figure 17: Train vs test set gen_roc. Left: membership train ↔ rosch test. Right: ifeval train ↔ ifeval OOD test.

6 Conclusions

1. **RankAlign works when the full method (s3) is used** — the NLL-V/G constraints are essential for preventing score explosion, especially on complex tasks like IFEval.
2. **TC-self training (s4) consistently provides the best results** across all metrics (gen ROC, spearman, concordance) and all model/task combinations.
3. **TC promotes generalization**: TC-trained models maintain their advantage on held-out test categories, while non-TC models (s3) can overfit to training patterns.
4. **Basic RankAlign (s2) is unreliable for complex tasks** — it can actually *hurt* performance (IFEval). The full method is needed.
5. **Score spread (gen_delta_mean) is a powerful training diagnostic** — values >1000 indicate instability. TC-self produces the tightest distributions.
6. **Concordance is a complementary metric to ROC AUC** — it directly measures the alignment between generator and validator ranking functions.